

10. Synthesisable SystemVerilog - Sequential logic

Sequential logic synthesis

- When writing code for sequential logic, use **always_ff** and **always_latch** blocks with incomplete conditional statements
- Do not mix complex combinational and sequential logic in one block; partition designs and use multiple blocks
- For state machines use edge-triggered flip-flops (**always_ff**)
- On new FPGAs, e.g. Altera Cyclone V, RAM is synchronous and requires **always_ff**. Use the FPGA vendor's code templates for RAM

10.1: Program Counter

Counters

```
parameter N = 8;  
logic [N-1:0] q;  
  
always_comb // or always_ff  
    q++;
```

Bad! Why?

```
parameter N = 8;  
logic [N-1:0] q,  
logic clk, n_reset;  
  
always_ff @(posedge clk,  
            negedge n_reset)  
    if (!n_reset)  
        q <= 0;  
    else  
        q <= q + 1;
```

Good

Program Counter

```
module pc #(parameter Psize = 6) // up to 64 instructions
(input logic clk, reset, PCincr, PCabsbranch, PCrelbranch,
 input logic [Psize-1:0] Branchaddr,
 output logic [Psize-1 : 0] PCout
);

logic[Psize-1:0] Rbranch; // temp variable for addition operand

always_comb // multiplexer to select next instruction offset Rbranch
    if (PCincr) // see always_ff block below
        Rbranch = 4; // add 4
    else Rbranch = Branchaddr; // add branch addr.

always_ff @ (posedge clk, posedge reset) // async reset
    if (reset) // reset
        PCout <= '0;
    else if (PCincr | PCrelbranch) // increment or branch relative
        PCout <= PCout + Rbranch; // 1 adder handles both
    else if (PCabsbranch) // absolute branch, load branch addr
        PCout <= Branchaddr;

endmodule // module pc
```

**Increment, relative branch,
absolute branch**

Not RISC-V, but similar

10.2: Synthesis Constraints

Timing Constraints

- Unlike (most) software, hardware has to meet specifications, other than functionality
 - Power, area, timing
 - Of these, timing is usually the most critical: silicon is cheap; but power is also a major constraint

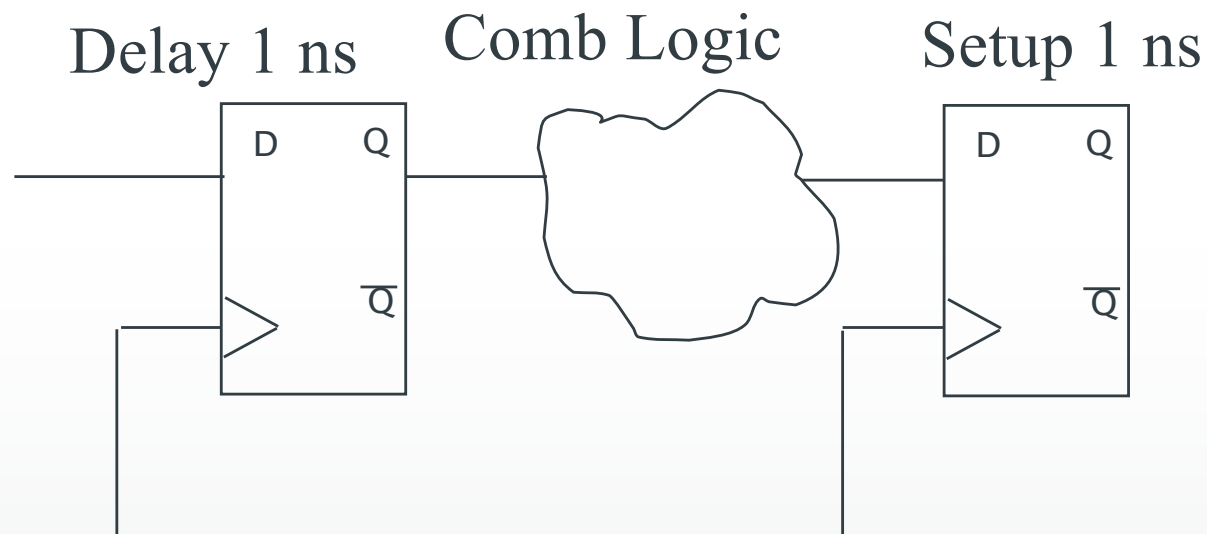
Clock Speed

- The maximum clock speed is defined by the choice of device.
- The desired clock speed can be set as a design objective:

```
create_clock -name "clock" -period 5.000ns  
[get_ports {clock}]
```

- Defines 200 MHz clock
- Synopsys .sdc file format is used by many tools
- Objective for Synthesis AND Place and Route

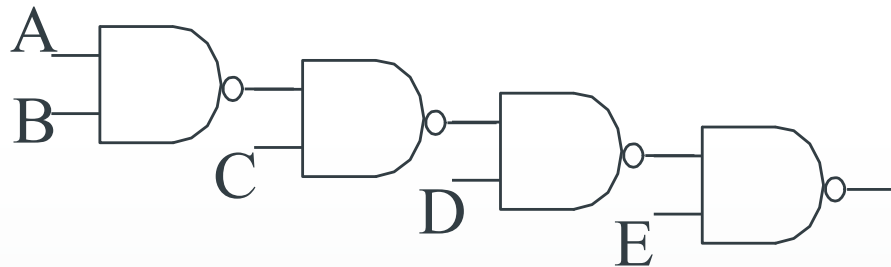
Timing Constraints



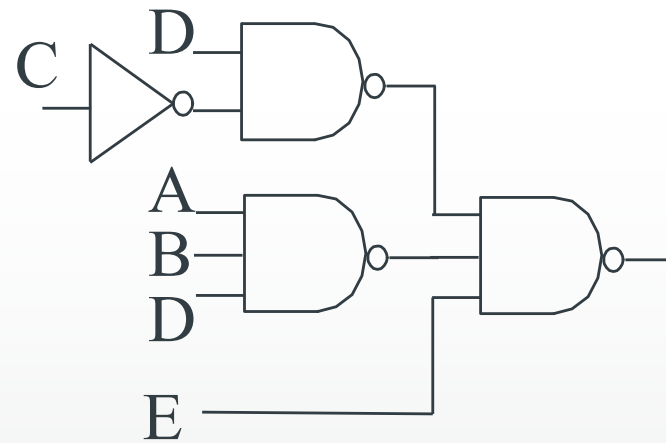
Clock period is 5 ns,
Clock Frequency is 200 MHz,
Max delay through Comb Logic is 3 ns

Logic Optimisation

- All optimisation, in terms of meeting constraints, assumes that there is more than one way to implement a function:

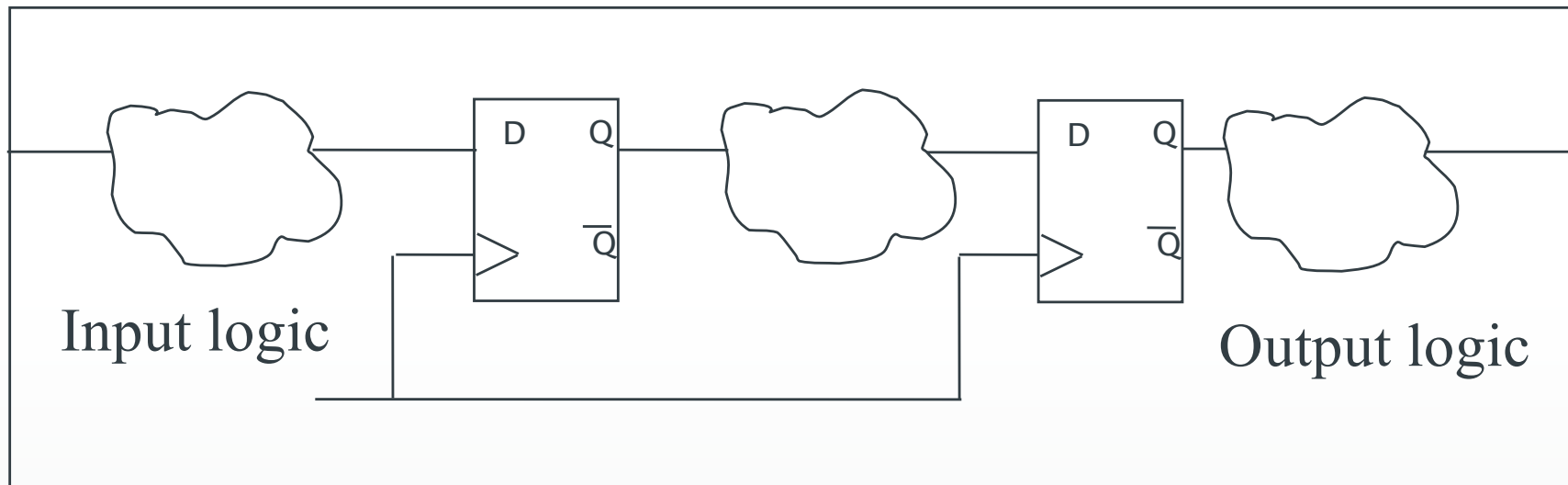


Assuming 1ns delay per gate,
4ns delay. Does not meet
constraint for 200 MHz operation



3ns delay. Meets constraint

Timing Constraints



External Input and Output delays can be specified:

```
set_input_delay 2.000ns -clock "clock"
```

Thus the maximum delay through the input logic is 2ns, allowing for 1ns setup time

Other Timing Constraints

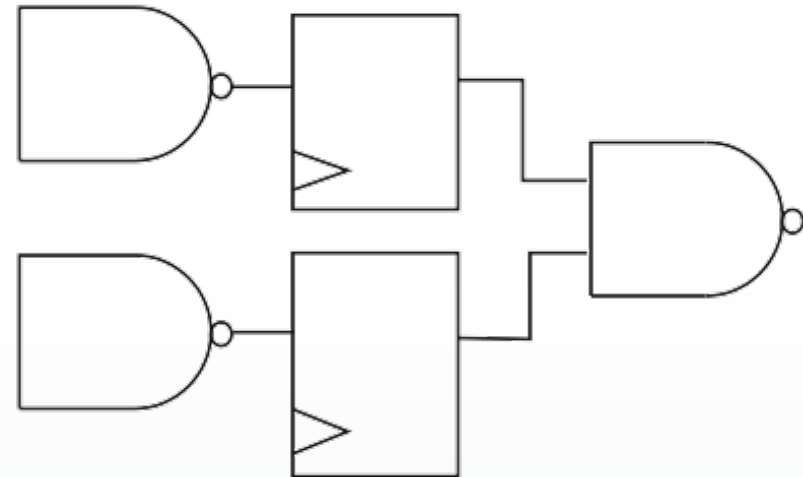
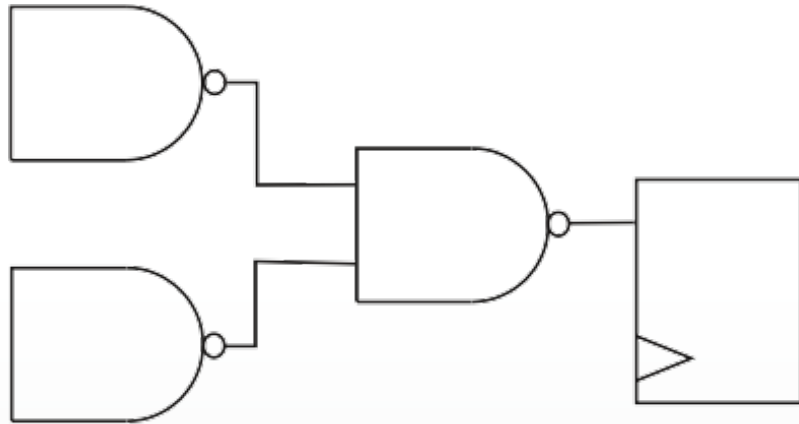
- There are a large number of possible constraints, e.g.
 - `set_clock_uncertainty`, defines uncertainty in the clock
 - `set_clock_skew`, defines uncertainty in the clock paths
 - `set_output_delay`, like `set_input_delay`
 - `set_false_path`, defines paths that cannot ever be logically active
 - `set_multicycle_path`, allows a signal to propagate over multiple cycles (dangerous)
- All used for *static* timing analysis in Vivado and similar tools (not simulation)

What if constraints not met?

- Do nothing? Metastable behaviour, or miss data completely
- Restructure logic, but may need significant changes
- Lose clock synchronisation
 - Large designs may need complex clock trees (i.e. multiple levels of buffers) to cause clock to arrive everywhere simultaneously
 - May need multiple clock domains. Common in big designs.

Retiming

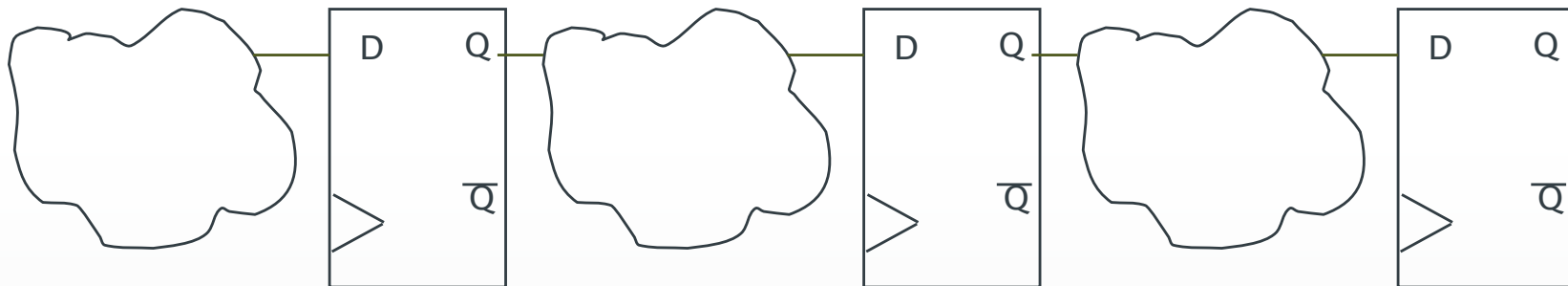
- Move logic between clock cycles:



- Balances delays between flip-flops. Can be done automatically, but makes cycle-by-cycle verification difficult.

Pipelining

- If delay through combinational logic is too great, logic can be divided:



- Clock speed is greater; time for one piece of data to go through (latency) is greater; throughput is also greater – one new result per clock cycle.
- Efficient if the pipeline is kept full
- Difficult to automate (though can be thought of as retiming)